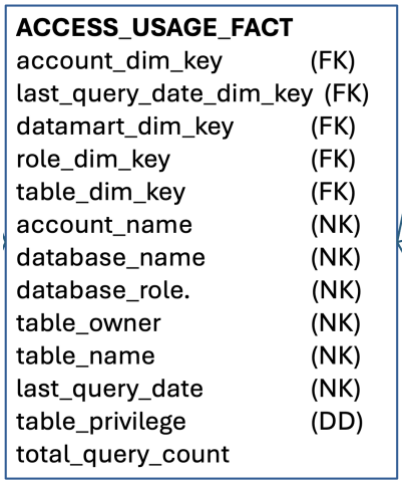
ACCESS_USAGE_FACT recipe

The ACCESS_USAGE_FACT I created is a periodic snapshot fact table, which contains last_query_date (when available) and total_query_count (when available).

The grain is “One record per database account each table_owner & table_name (and privilege) the account has access to”. Note that the database, datamart, and role are not mentioned, that is because they are a higher grain than the table access.



A few things to not about this fact table.

1) Technically, it should have a snapshot date, and associated key. Since we can only run one snapshot, and the most recent data is a few months old, I am leaving it off for the POC. Including this would certainly be closer to a production implementation.
2) The total_query_count is a compromise, given I only have one snapshot. In a production deployment, where you would have many snapshots, you would want a fully additive fact, maybe just query_count, representing the number of queries since the last snapshot. This would be fully additive.
3) Note all of the (NK)s in this fact table. The Kimball examples generally do not include the natural keys in the fact table, only the dimension tables. Including the NKs in the fact table diagrams is not required, However, since you will need to understand the relationship, and the stage table, prior to your fact table, will need the natural keys for surrogate key mapping, I have included them. It will help inform the ETL development. You may include them in your fact table, or remove from fact table. Your choice.
4) Note the name of this fact table, I am trying to convey its business purpose and supporting both the access related requirements and the usage requirement.

Source-to-target mappings:

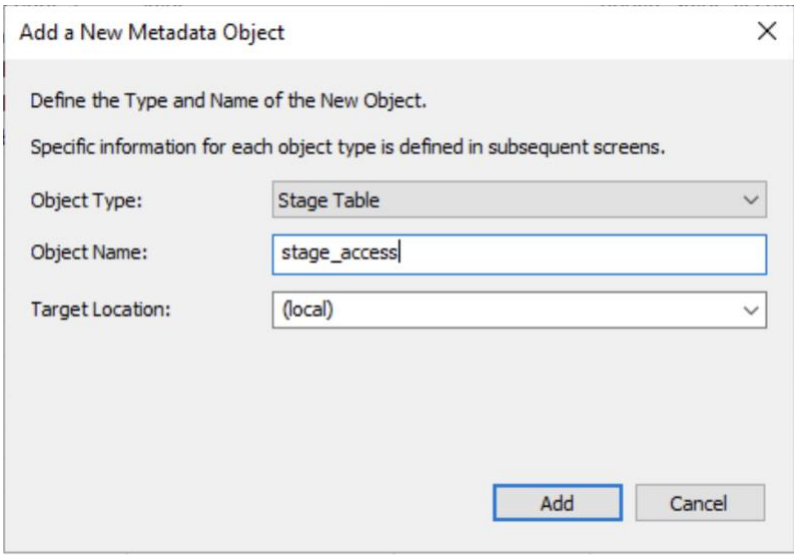
Source table	Source column	Target table	Target column	Transformation / notes
Oracle_role_privs, Oracle_table_audit	Username, Username	DW_Access_usage_fact	Account_name	
Oracle_role_privs, Oracle_table_audit	Database_name, Database_name	DW_Access_usage_fact	Database_name	
Oracle_roles_to_tables	Database_role	DW_Access_usage_fact	Database_role	
Oracle_roles_to_tables, Oracle_table_audit	Owner, Owner	DW_Access_usage_fact	Table_owner	
Oracle_roles_to_tables, Oracle_table_audit	Table_name, Obj_name	DW_Access_usage_fact	Table_name	
Oracle_roles_to_tables, Oracle_table_audit	Privilege, Action_name	DW_Access_usage_fact	Table_privilege	
Oracle_table_audit	Query_date	DW_Access_usage_fact	Last_query_date	Max(query_date)
Oracle_table_audit	Query_count	DW_Access_usage_fact	Total_query_count	Sum(query_count)

Load Table:

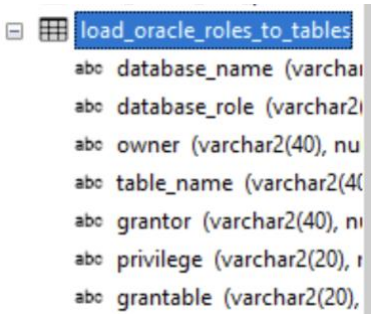
Create a load tables for oracle_role_privs , oracle_roles_to_tables, oracle_table_audit. Remember to use your audit_src connection for load tables. *Note that you may have the load tables already, if someone on your team built them as part of one of the other recipes*.

Stage Tables:

1) In left pane, double click “Stage Table”, so stage table is the focus of the middle pane.
2) In right pane, change your connection to warehouse connection.
3) In right pane, select load_oracle_role_privs, drag to middle pane
4) In the “Add a New Metadata Object” screen, name the object “Stage_acces”, then click Add. *See screenshot below.*



5) For the "Stage Table stage_access" screen, simply click OK.
6) In the right pane, double click "load_oracle_roles_to_tables" to expand it. You need to see the column names.



7) Select column owner and drag to middle pane. Repeat for table_name and privilege columns. Note that I am not bringing the database_name and database_role columns. I already have them from the load_oracle_role_privs table. I am also not grabbing all of the columns, such as grantor and grantable. You could, it depends on your data model. What did you include in your model?
8) Middle pane should look similar to this:

stage_access Columns

Column Name	Display Name	Data Type	Source Table	Source Column	F
database_name	database name	varchar2(30)	load_oracle_role_p...	database_name	
username	username	varchar2(40)	load_oracle_role_p...	username	
granted_role	granted role	varchar2(40)	load_oracle_role_p...	granted_role	
admin_option	admin option	varchar2(30)	load_oracle_role_p...	admin_option	
owner	owner	varchar2(40)	load_oracle_roles_t...	owner	
table_name	table name	varchar2(40)	load_oracle_roles_t...	table_name	
privilege	privilege	varchar2(20)	load_oracle_roles_t...	privilege	
dss_create_time	dss create time	date		dss_create_time	
dss_update_time	dss update time	date		dss_update_time	

9) I renamed the following columns to match my data model:
   Username to account_name
   Granted_role to database_role
   Owner to table_owner
   Privilege to table_privilege

stage_access Columns

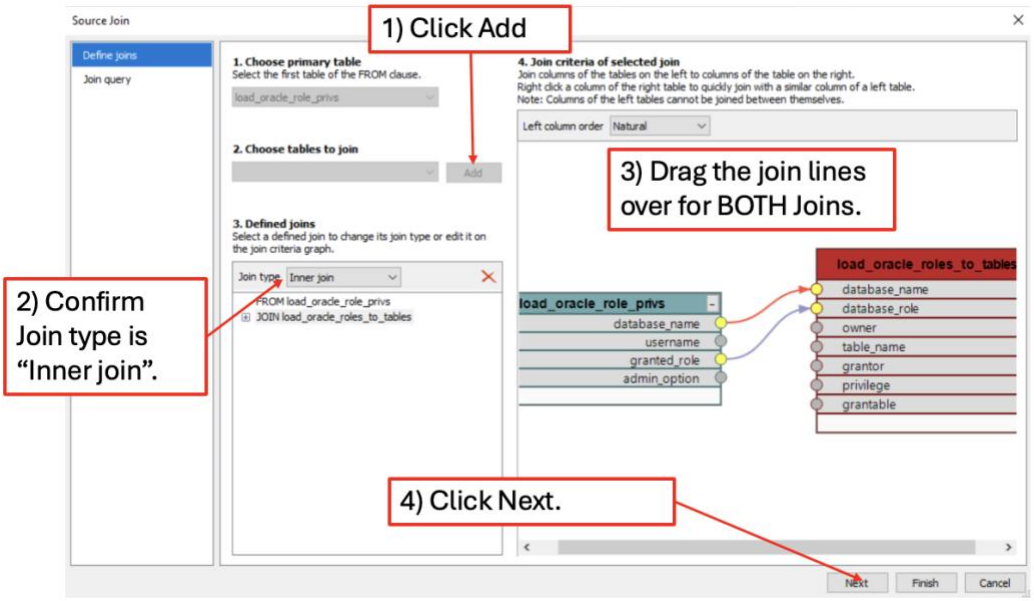
Column Name	Display Name	Data Type	Source Table	Source Column
database_name	database name	varchar2(30)	load_oracle_role_p...	database_name
account_name	account name	varchar2(40)	load_oracle_role_p...	username
database_role	database role	varchar2(40)	load_oracle_role_p...	granted_role
admin_option	admin option	varchar2(30)	load_oracle_role_p...	admin_option
table_owner	table owner	varchar2(40)	load_oracle_roles_t...	owner
table_name	table name	varchar2(40)	load_oracle_roles_t...	table_name
table_privilege	table privilege	varchar2(20)	load_oracle_roles_t...	privilege
dss_create_time	dss create time	date		dss_create_time
dss_update_time	dss update time	date		dss_update_time

10) For my data model, I did not include admin_option, so I am dropping this column. Right click on admin_option then delete column. Note that if your data model includes this column, leave it in place.

stage_access Columns

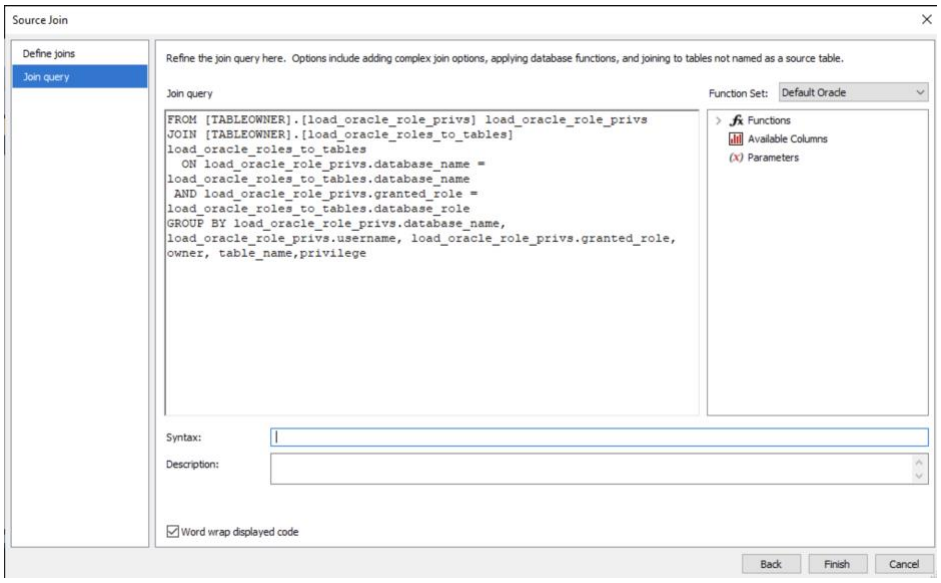
Column Name	Display Name	Data Type	Source Table	Source Column
database_name	database name	varchar2(30)	load_oracle_role_p...	database_name
account_name	account name	varchar2(40)	load_oracle_role_p...	username
database_role	database role	varchar2(40)	load_oracle_role_p...	granted_role
table_owner	table owner	varchar2(40)	load_oracle_roles_t...	owner
table_name	table name	varchar2(40)	load_oracle_roles_t...	table_name
table_privilege	table privilege	varchar2(20)	load_oracle_roles_t...	privilege
dss_create_time	dss create time	date		dss_create_time
dss_update_time	dss update time	date		dss_update_time

11) In left pane, right click on stage_access, then select “create(recreate)”.
12) In left pane, right click on stage_access, then select “code > Rebuild Update Procedure”.
13) For the “Define Stage Procedure Type” screen, simply click “Set”.
14) For the “Select Parameters”, simply click OK.
15) Define the two needed joins. *See screenshot below for details*.



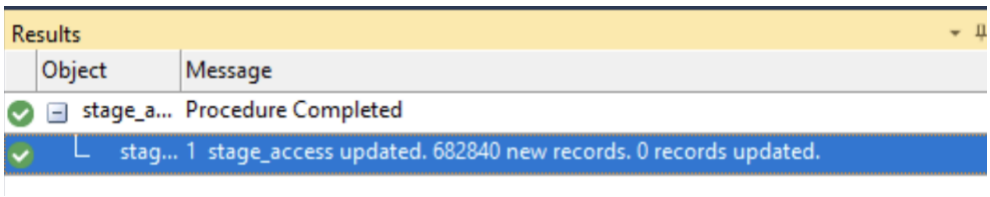
16) In the "Join query" code box, add the following code:

GROUP BY load_oracle_role_privs.database_name, load_oracle_role_privs.username, load_oracle_role_privs.granted_role, owner, table_name,privilege



17) Click "Finish".

18) In left pane, right click stage_access, then select "execute update procedure".

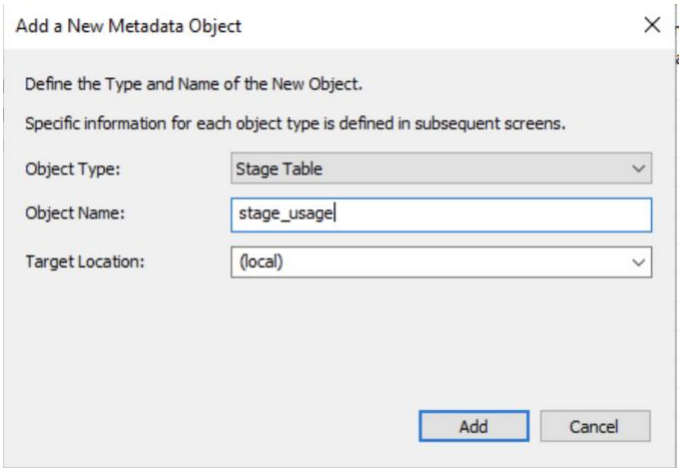


19) In left pane, right click stage_access, then select “display data”. How does the data look? You now have a table containing all of the security mappings. Basically, one snapshot for a periodic snapshot fact table containing all of the security mappings.

STAGE_USAGE:

Now we need to build a table, off the audit data, aggregated to the same grain as the stage_access table.

1) In left pane, double click “Stager Table”, so Stage Table becomes the focus of the middle pane.
2) In right pane, refresh your data warehouse connection by clicking the blue sunglasses.
3) In right pane, select load_oracle_table_audit and drag to middle pane.
4) For the “Add a New Metadata Object” screen, rename the Object name to “stage_usage”. Then click Add.



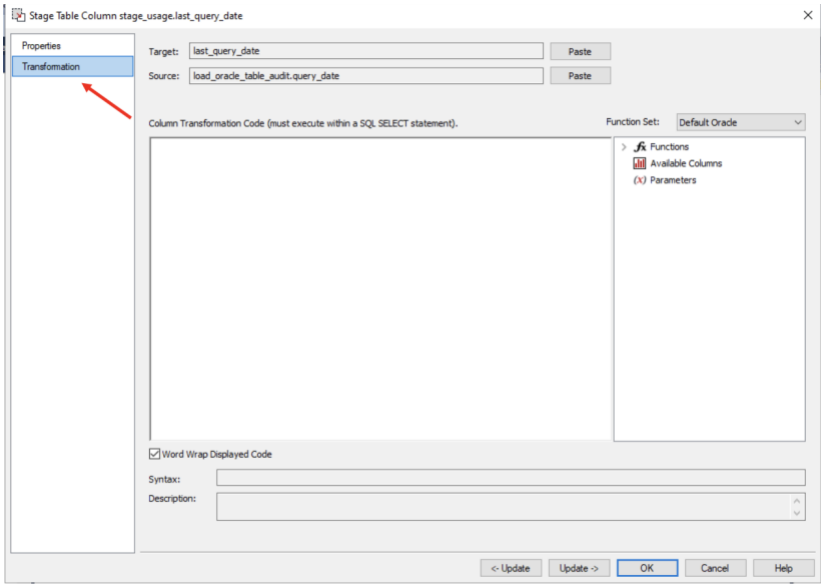
5) For the “Stage Table stage_usage” screen, simply click OK.
6) Next I renamed the following columns to make this stage table consistent with my data model:
   Username to account_name
   Query_date to last_query_date
   Action_name to table_privilege

Owner to table_owner
Obj_name to table_name
Query_count to total_query_count

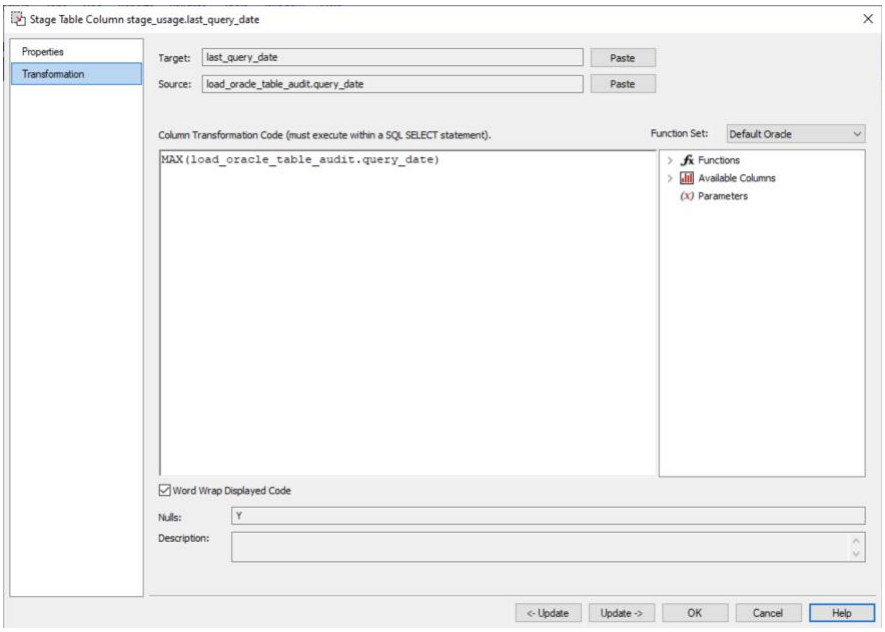
stage_usage Columns

Column Name	Display Name	Data Type	Source Table	Source Column
database_name	database name	varchar2(30)	load_oracle_table_...	database_name
account_name	account name	varchar2(40)	load_oracle_table_...	username
last_query_date	last query date	date	load_oracle_table_...	query_date
table_privilege	table_privilege	varchar2(20)	load_oracle_table_...	action_name
table_owner	table_owner	varchar2(40)	load_oracle_table_...	owner
table_name	table name	varchar2(40)	load_oracle_table_...	obj_name
total_query_count	total query co...	number(38)	load_oracle_table_...	query_count
dss_create_time	dss create time	date		dss_create_time
dss_update_time	dss update time	date		dss_update_time

7) In middle pane, double click on the last_query_date column, then select the "Transformation" tab in the upper left corner. See screenshot below.

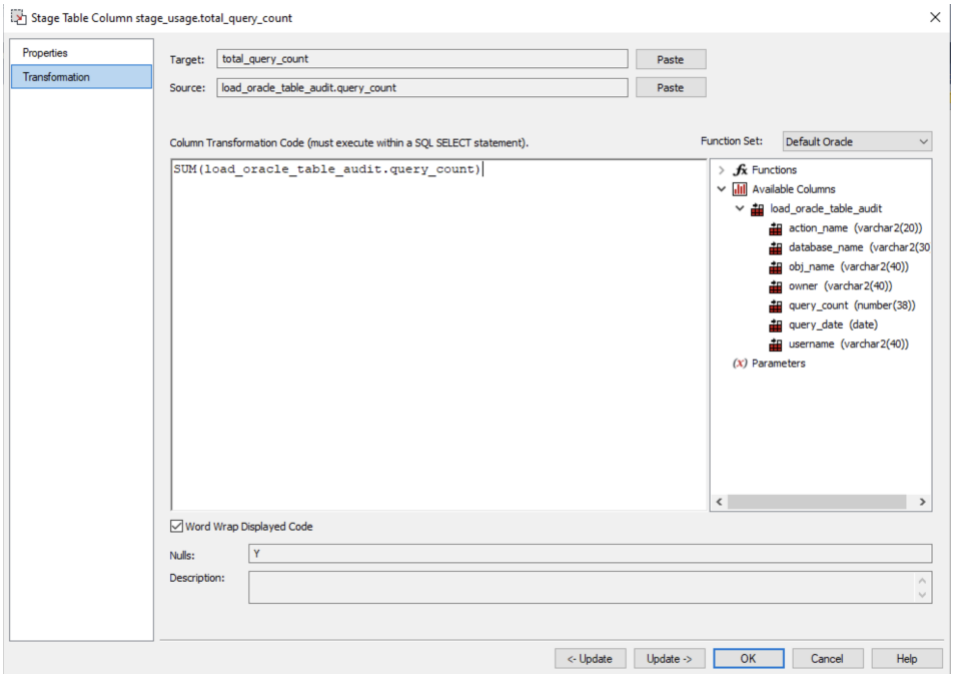


8) Paste the following code into the transformation code window.
MAX(load_oracle_table_audit.query_date)

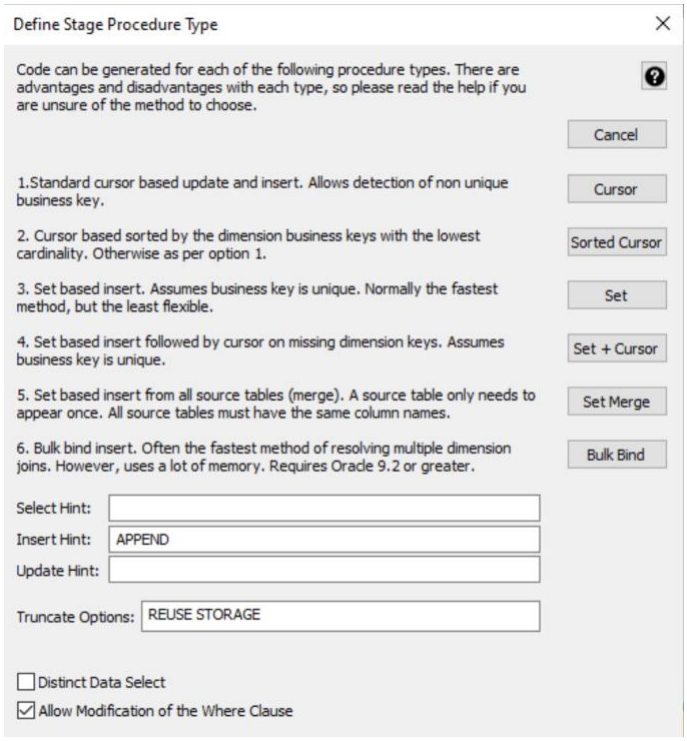


9) Click OK
10) In middle pane, double click the total_query_count column, then click the "transformation" tab in the upper left corner.
11) In the transformation code window, past the following code:
SUM(load_oracle_table_audit.query_count)

Note this is a SUM(), not a COUNT().

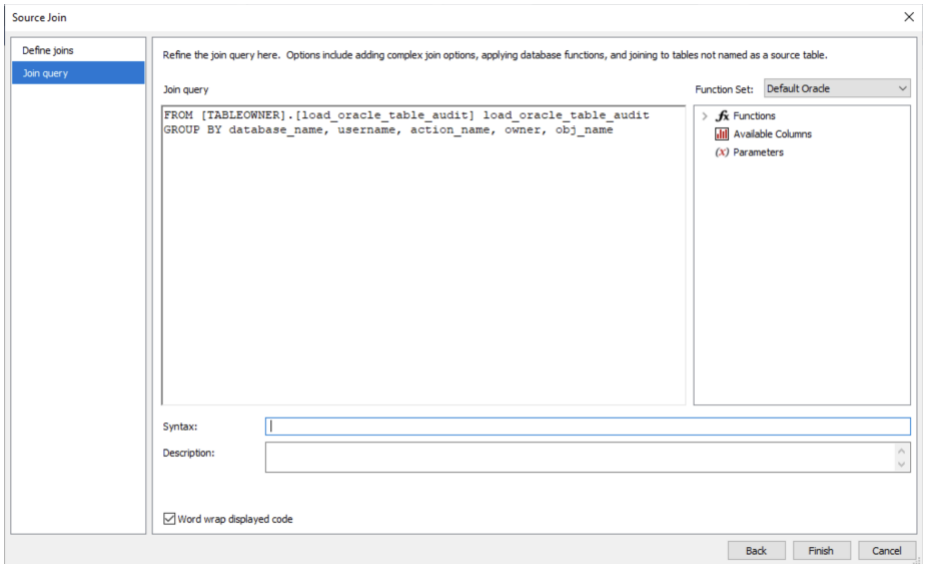


12) Click OK.
13) In left pane, right click stage_usage, then select “create(recreate)”.
14) In left pane, right click stage_usage, then select “code > rebuild update procedure”.
15) For the “Define Stage Procedure Type” screen, check the box for “Allow Modification of the Where Clause”, then select “Set”.

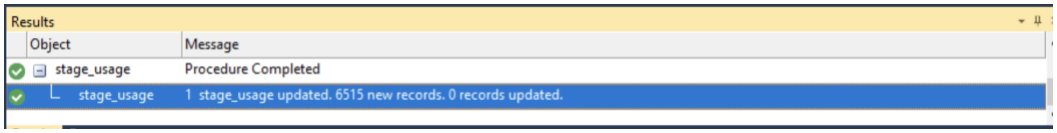


16) For the “Select Parameters” screen, click OK.
17) In the “Source Join” screen, paste the following code, below the from statement.

GROUP BY database_name, username, action_name, owner, obj_name



18) Click Finish.
19) In left pane, right click stage_usage, then select “execute update procedure”.



20) In left pane, right click stage_usage, then select “display data”. How does the data look?

STAGE_ACCESS_USAGE:

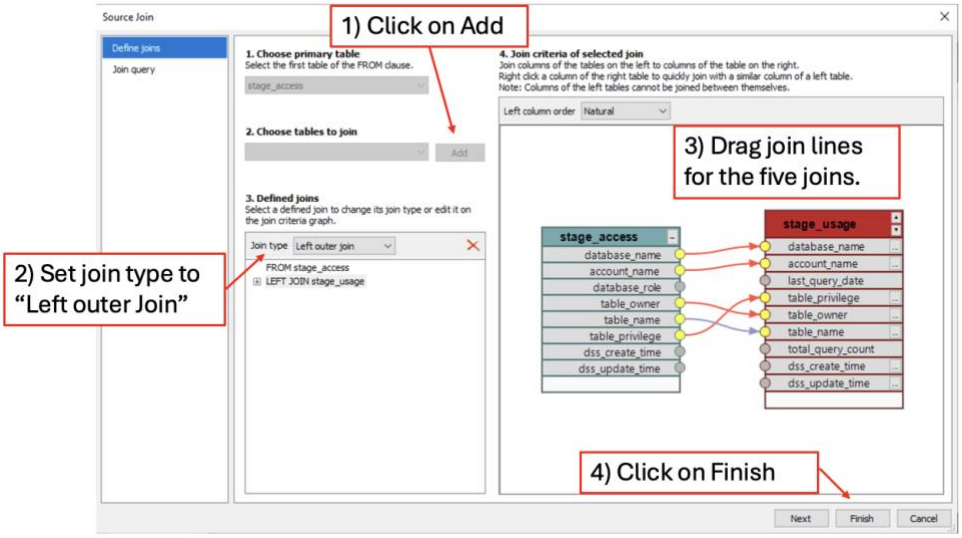
We will now merge the usage data with the access data. Since some of the access is not used, we will need to outer join stage_access to stage_usage.

1) In left pane, double click “Stage Table”, so Stage Table becomes the focus of the middle pane.
2) In the right pane, click the blue sunglasses to refresh your data warehouse connection.
3) In right pane, select stage_access and drag to the middle pane.
4) In the “Add a New Metadata Object” screen, change object Name to “stage_access_usage”, then click Add.
5) For the “Stage Table stage_access_usage” screen, simply click “OK”.
6) In right pane, double click stage_usage table to expand. You should see columns.
7) In right pane, select last_query_count and drag to middle pane.
8) In right pane, select total_query_count and drag to middle pane.

stage_access_usage Columns

Column Name	Display Name	Data Type	Source Table	Source Column
database_name	database name	varchar2(30)	stage_access	database_name
account_name	account name	varchar2(40)	stage_access	account_name
database_role	database role	varchar2(40)	stage_access	database_role
table_owner	table owner	varchar2(40)	stage_access	table_owner
table_name	table name	varchar2(40)	stage_access	table_name
table_privilege	table privilege	varchar2(20)	stage_access	table_privilege
last_query_date	last query date	date	stage_usage	last_query_date
total_query_count	total query co...	number(38)	stage_usage	total_query_count
dss_create_time	dss create time	date		dss_create_time
dss_update_time	dss update time	date		dss_update_time

9) In left pane, right click stage_access_usage, then select “create(recreate)”.
10) In left pane, right click stage_access_usage, then select “code > rebuild update procedure”.
11) In the “Define Stage Procedure Type” screen, click “Set”.
12) For the “Select Parameters”, simply click OK.
13) In the “Source Join, you will be defining 5 outer joins between the stage_access and stage_usage tables. Follow steps in screenshot below.



14) In left pane, right click stage_access_usage, then select “execute update procedure”.
15) In left pane, right click stage_access_usage, then select “display data”. How does the data look? Notice how many records have null for last_query_date and total_query_count. There appears to be a lot of access not being used. At least during this 3 month period. Note that we should set a default value of 0 for the total_query_count when null. We will do that shortly.

ACCESS_USAGE_FACT recipe

You will need the following dimensions and dimension view before you can proceed with the surrogate key mapping.

ACCOUNT_DIM
DATAMART_DIM
ROLE_DIM
TABLE_DIM
LAST_QUERY_DATE_DIM

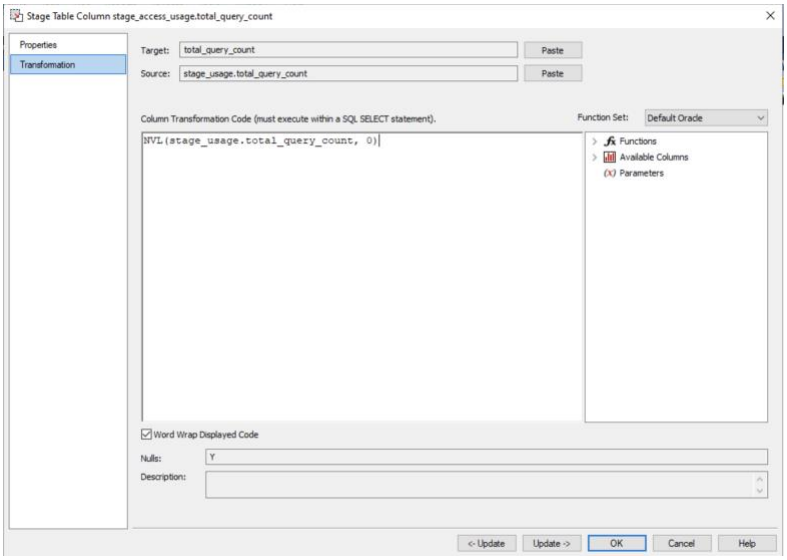
1)In left pane, right click stage_access_usage, then select display columns.

2) in middle pane, double click total_query_count.

3) in the upper left corner, click on the “Transformation” tab.

4) paste the following code in the “Transformation code” box.

NVL(stage_usage.total_query_count, 0)



5) Click OK.
6) In the right pane, click the blue sunglasses to refresh data warehouse connection.
7) In the right pane, double click on account_dim, to expand and show columns.
8) In the right pane, select the account_dim_key and drag to the middle pane.
9) Repeat the above process pulling the remaining dimension keys into the middle pane
    a. Datamart_dim_key
    b. Dim_last_query_dim
    c. Role_dim_key

d. Table_dim_key

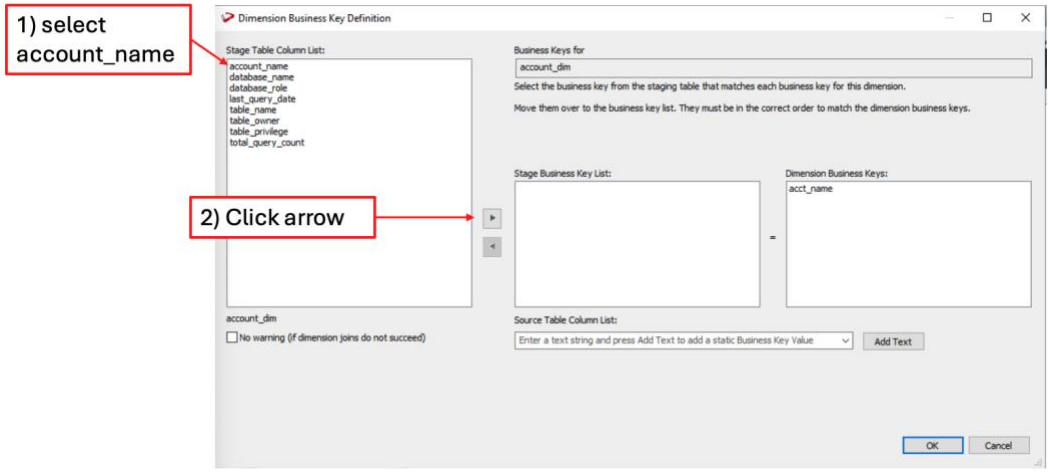
10) I rearranged the columns to match my data model. The end result looks like this:

stage_access_usage Columns

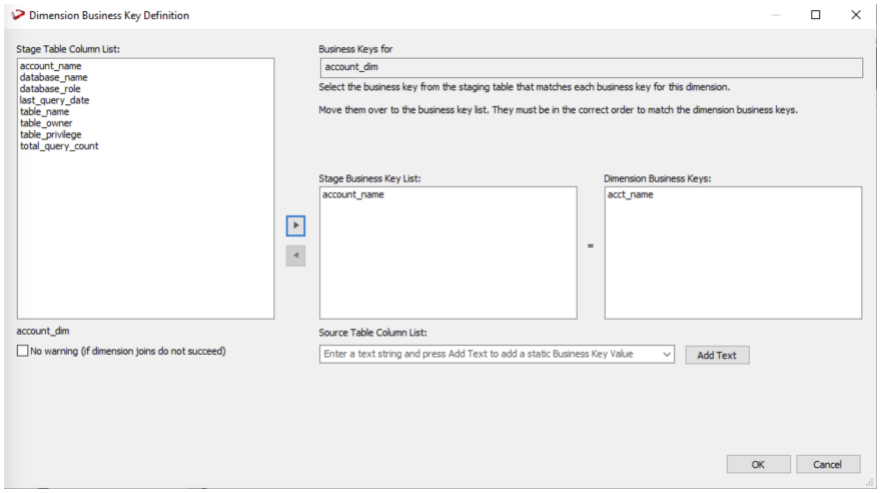
Column Name	Display Name	Data Type	Source Table	Source Column
account_dim_key	account dim key	integer	account_dim	account_dim_key
dim_last_query_date_dim_key	dim last query ...	integer	last_query_date_dim	dim_last_query_da...
datamart_dim_key	datamart dim ...	integer	datamart_dim	datamart_dim_key
role_dim_key	role dim key	integer	role_dim	role_dim_key
table_dim_key	table dim key	integer	table_dim	table_dim_key
account_name	account name	varchar2(40)	stage_access	account_name
database_name	database name	varchar2(30)	stage_access	database_name
database_role	database role	varchar2(40)	stage_access	database_role
table_owner	table owner	varchar2(40)	stage_access	table_owner
table_name	table name	varchar2(40)	stage_access	table_name
table_privilege	table privilege	varchar2(20)	stage_access	table_privilege
last_query_date	last query date	date	stage_usage	last_query_date
total_query_count	total query co...	number(38)	stage_usage	total_query_count

Note my dim_last_query_date_dim_key. It should be simply last_query_date_dim_key. Rather than rebuild my last_query_date_dim dimension view, to fix the key name, I will simply adjust my data model for this key naming.

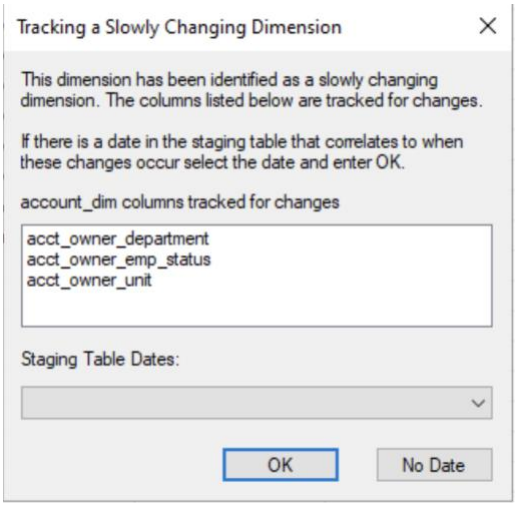
11) In left pane, right click stage_access_usage, then select “create(recreate)”.
12) You will likely get a screen warning about losing data, blah blah. Simply click OK. We will be reloading anyway.
13) You will next get a warning about no keys defined, this is because we have not rebuilt the code yet, so simply click “Yes” to proceed.
14) You will receive additional error messages about failing to compile your update_stage_access_usage procedure. Simply click OK on any of these error. The reason the code will not compile is the code is out of since with our new table definition. We will fix this in a minute.
15) In the left pane, right click stage_access_usage, then select “code > Rebuild update_stage_access_usage”.
16) For the “Define Stage Procedure Type” screen, click “Set”.
17) For the “Select Parameters”, simply click OK.
18) For the Source Join screen, click Finish.
19) For the first “Dimension Business Key Definition” screen, I see that I have a different business key (NK) name for my account_dim, so in far left select “account_name” then the right arrow. See screenshot below.



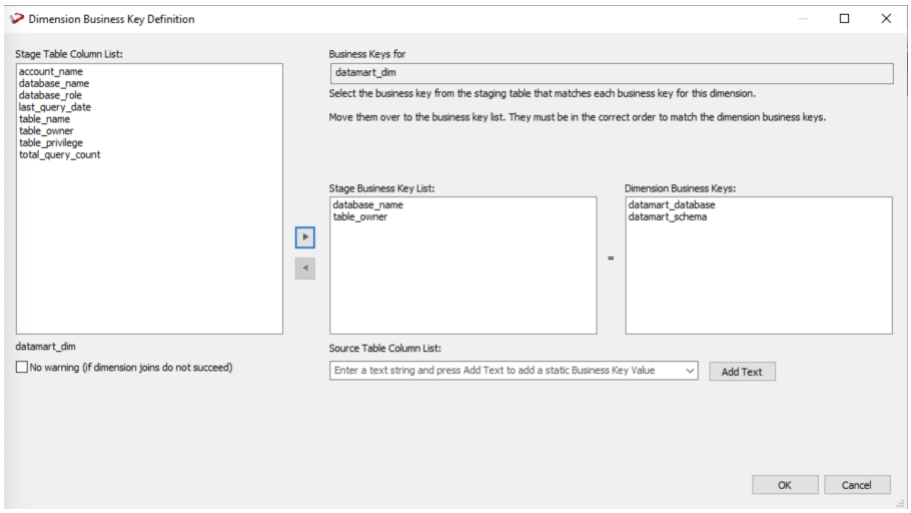
20) Click OK.



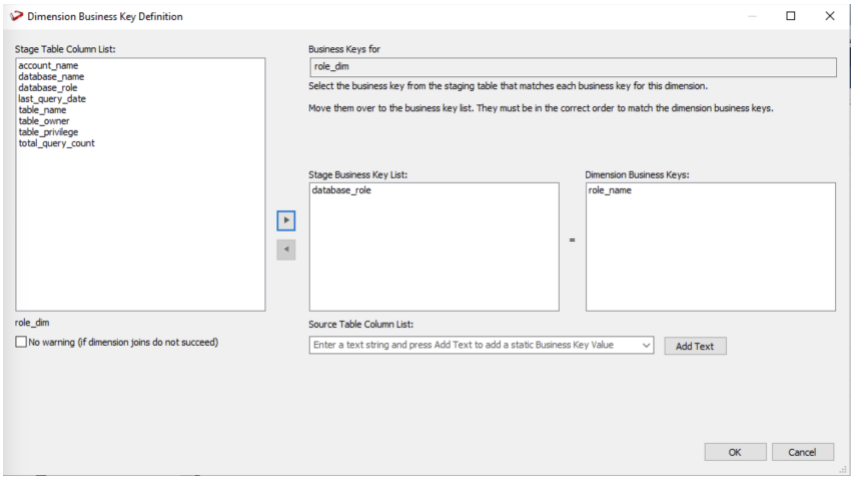
21) For the “Tracking a Slowly Changing Dimension” screen, there is no date, so simply click “No Date”.



22) For the remaining Business Key Definition screens, if both keys are showing, simply click OK.
23) In the “Dimension Business Key Definition” screen for “datamart_dim” you will see there are two business keys on the right side: datamart_database & datamart_schema. Similar to step 20 above, select database_name, then table_owner. Should look like screenshot below.

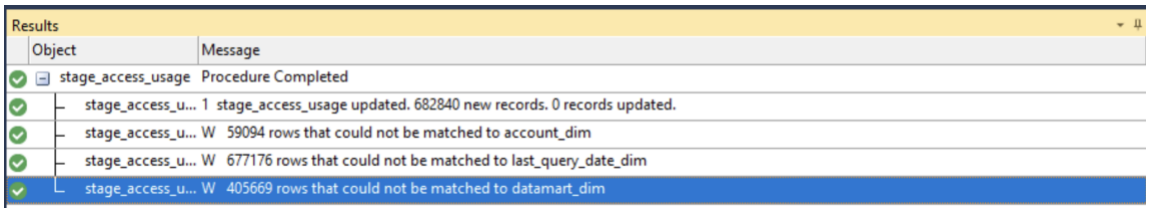


24) Click OK.
25) In the “Dimension Business Key Definition” screen for role_dim, set the stage business key to: database_role, then click OK.



26) In left pane, right click stage_access_usage, then click execute update procedure.

ACCESS_USAGE_FACT recipe



27) In left pane, right click stage_access_usage, then select "display data". How does the data look? Is the default 0 for total_query_count transformation working? You will notice a lot of zero key values for dim_last_query_date & datamart_dim. This is ok, but you should have some non-zeros. The best way to validate your data is with SQL, you can access via SQL Developer.

ACCESS_USAGE_FACT:

1) In left pane, double click "Fact Table", to make Fact Table the focus of middle pane.
2) In right pane, click the blue sunglasses to refresh your data warehouse connection.
3) In right pane, select the stage_access_usage table and drag to middle pane.
4) In the "Add a New Metadata Object" screen, rename table to "access_usage_fact", then click Add.

5) For the "Fact Table access_usage_fact" screen, simply click OK.

6) For the "Create Database Table" screen, click "Close".

7) I dropped the two dss_ columns from my fact table. Not necessary, but I felt like it. Here's a screenshot of my table columns:

access_usage_fact Columns

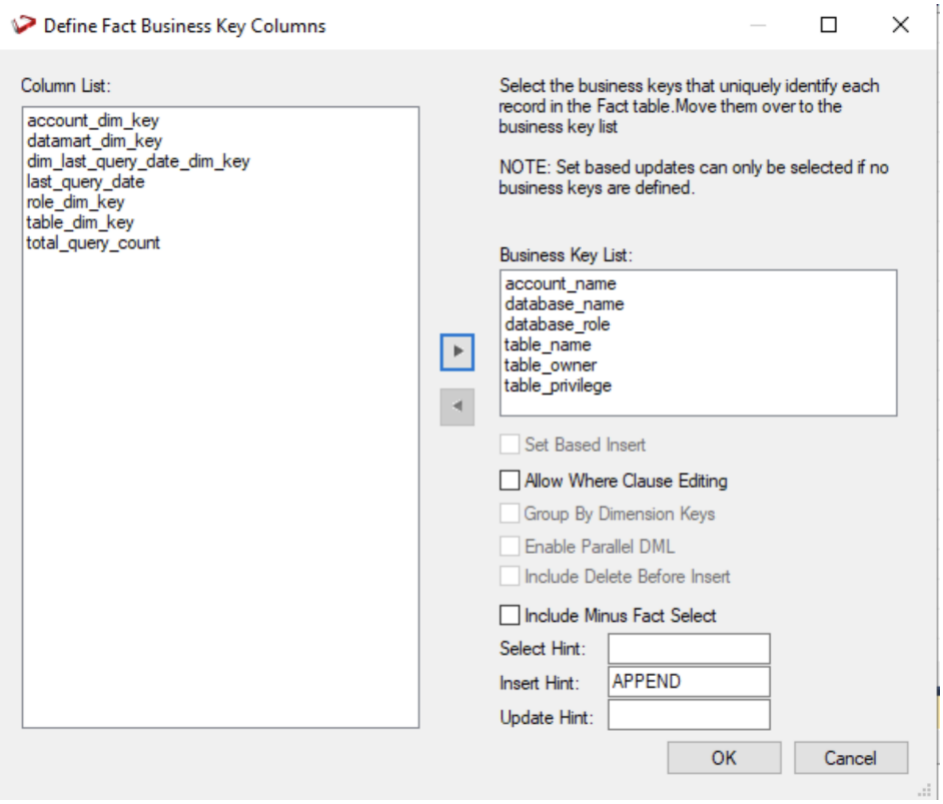
Column Name	Display Name	Data Type	Source Table	Source Column
account_dim_key	account dim key	integer	account_dim	account_dim_key
dim_last_query_date_dim_key	dim last query ...	integer	last_query_date_dim	dim_last_query_da...
datamart_dim_key	datamart dim ...	integer	datamart_dim	datamart_dim_key
role_dim_key	role dim key	integer	role_dim	role_dim_key
table_dim_key	table dim key	integer	table_dim	table_dim_key
account_name	account name	varchar2(40)	stage_access_usage	account_name
database_name	database name	varchar2(30)	stage_access_usage	database_name
database_role	database role	varchar2(40)	stage_access_usage	database_role
table_owner	table owner	varchar2(40)	stage_access_usage	table_owner
table_name	table name	varchar2(40)	stage_access_usage	table_name
table_privilege	table privilege	varchar2(20)	stage_access_usage	table_privilege
last_query_date	last query date	date	stage_access_usage	last_query_date
total_query_count	total query co...	number(38)	stage_access_usage	total_query_count

8) In left pane, right click access_usage_fact, then select "create(recreate)".

9) In left pane, right click access_usage_fact, then select “code > rebuild update procedure”.

10) If you delete the dss_ columns like I did, you may receive a warning message about bad carma and stuff. Simply click yes to proceed.

11) For the “Define Fact Business key Columns” screen, there are A LOT of business keys. Drag over: account_name, database_name, database_role, table_name, table_owner, table_privilege. See screenshot below.



12) Click OK.

13) Cross your fingers for luck.

14) In left pane, right click access_usage_fact, then select “execute update procedure”.

Warning: the fact table load takes a while. My code ran 4.5 minutes. If you happen to be running your code with the INFO4555 database is busy, like the night before the deadline, it may take much longer.

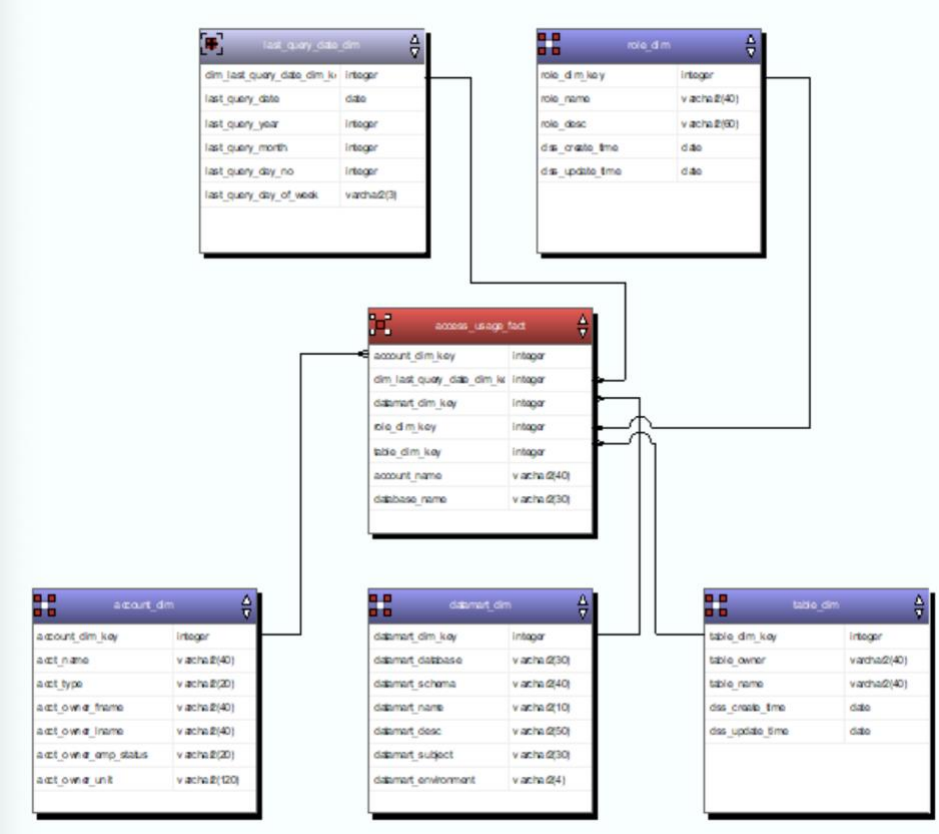
ACCESS_USAGE_FACT recipe

Results

Object	Message
stage_access_usage	Procedure Completed
access_usage_fact	SQL Statement completed.
access_usage_fact	Procedure 'update_access_usage_fact' for Fact Table 'access_usage_fact' is rebuilt and recompiled.
access_usage_fact	Procedure Completed
access_usage_f... 1	access_usage_fact updated. 682840 new records. 0 records updated.

16) In left pane, right click access_usage_fact, then select “display data”. Should look similar to the data in your stage_access_usage table.

Congratulations! You have a populated data model. Time to connect with Tableau and begin playing with the data.



ACCESS_USAGE_FACT recipe

